DASH-02: Dashboard Hierarchy

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

Not every stakeholder needs the same view. A well-designed dashboard strategy uses a three-tier hierarchy — Executive, Operations, and Engineering — where each tier serves a different audience with appropriate information density. This notebook defines each tier, explains how to select metrics and refresh cadence for each, and provides sample DQL queries representative of each level.

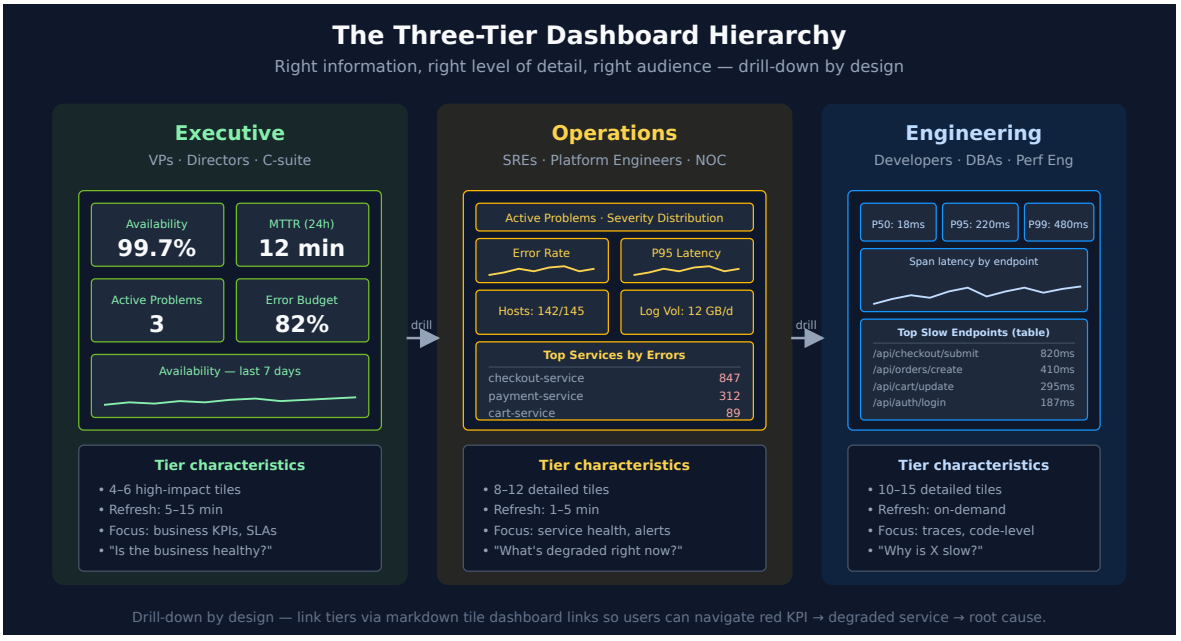
Table of Contents

1. The Three-Tier Model
2. Executive Tier
3. Operations Tier
4. Engineering Tier
5. Information Density by Tier
6. Refresh Cadence Recommendations
7. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`storage:logs:read`, `storage:metrics:read`, `storage:events:read`, `storage:spans:read`
Prior Reading	DASH-01: Dashboard Fundamentals

1. The Three-Tier Model



The three-tier hierarchy ensures every stakeholder gets the right information at the right level of detail.

Tier	Audience	Focus	Tile Count	Refresh
Executive	VPs, Directors, C-suite	Business KPIs, SLAs, high-level health	4-6	5-15 min
Operations	SREs, Platform Engineers, NOC	Service health, alerts, infrastructure	8-12	1-5 min

Tier	Audience	Focus	Tile Count	Refresh
Engineering	Developers, DBAs, Performance Engineers	Traces, code-level metrics, deep dives	10-15	On-demand

Navigation Pattern

Design dashboards so users can drill down through the tiers:

1. **Executive** dashboard shows a red SLA indicator
2. Click through to **Operations** dashboard to see which service is degraded
3. Click through to **Engineering** dashboard for trace-level root cause

Use dashboard links in markdown tiles to connect the tiers.

2. Executive Tier

Executive dashboards answer: **"Is the business healthy?"**

Characteristics

- **Minimal tiles** — 4-6 maximum
- **Single-value and trend** tiles dominate
- **Traffic light colors** — green/yellow/red for instant comprehension
- **No technical jargon** — "Availability" not "HTTP 200 ratio"
- **Longer time ranges** — daily/weekly trends, not minute-by-minute

Sample KPIs

KPI	Query Approach	Tile Type
Overall availability %	Successful ÷ total requests, or an SLO — **not** problem downtime, which overlaps and over-counts (DASH-03 § 2)	Single value
Mean time to resolve	Closed problem duration average — how long problems stayed open, not time to human resolution (ADOPT-03 § 3)	Single value

KPI	Query Approach	Tile Type
Active problem count	Count of open detected problems	Single value
Problem trend (7 day)	Problem count timeseries	Line chart

Example: Weekly Problem Trend

```
// Weekly problem trend — suitable for executive line chart
// time:event.start bins each problem on the day it opened. The default (timestamp) is the
// problem's last update, which counts old problems again on the day they were touched.
fetch dt.davis.problems, from:-7d
| makeTimeseries problem_count = count(), interval:24h, time:event.start
```

Example: Active Problem Count (Single Value)

```
// Active problems — single value tile for executive dashboard
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| filter dt.davis.is_duplicate == false
| summarize active_count = countDistinct(event.id)
```

3. Operations Tier

Operations dashboards answer: **"What needs attention right now?"**

Characteristics

- **8-12 tiles** with mixed chart types
- **Real-time focus** — 1-2 hour time windows
- **Service-centric** — grouped by service or application
- **Alert integration** — active problems and events prominent
- **Top-N lists** — worst offenders by error rate, latency, volume

Sample Metrics

Metric	Query Approach	Tile Type
Service response time	Span duration by service	Line chart
Error rate by service	Error count / total count	Bar chart
Log volume by level	Log count timeseries	Stacked area
Top 5 problem types	detected problem aggregation	Table
Infrastructure CPU	Host CPU timeseries	Line chart

Example: Service Error Rate

```
// Error rate by service — operations bar chart
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {total = count(), errors = countIf(span.status_code == "error")},
by:{dt.entity.service}
| fieldsAdd error_rate = 100.0 * errors / total
| sort error_rate desc
| limit 10
```

Example: Log Volume by Level

```
// Log volume trend by severity — operations stacked area chart
fetch logs, from:-2h
| filterOut loglevel == "NONE"
| makeTimeseries log_count = count(), interval:5m, by:{loglevel}
```

4. Engineering Tier

Engineering dashboards answer: **"Why is this happening and where exactly?"**

Characteristics

- **10-15 tiles** with high data density
- **Deep-dive focus** — specific services, endpoints, database queries

- **Filter-heavy** — variables for service, environment, namespace
- **Table tiles** — detailed data for investigation
- **On-demand refresh** — engineers trigger when investigating

Sample Metrics

Metric	Query Approach	Tile Type
Slowest endpoints	Span p95 by HTTP route	Table
Database query time	DB span duration by statement	Table
Trace error details	Span attributes for failed traces	Table
Deployment impact	Before/after latency comparison	Line chart

Example: Top 10 Slowest Endpoints

```
// Top 10 slowest endpoints by p95 response time
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(http.route)
| summarize {p95_ms = percentile(duration, 95) / 1ms, request_count =
count()}, by:{http.route, dt.entity.service}
| sort p95_ms desc
| limit 10
```

Example: Database Performance by System

```
// Database response time by system — engineering detail table
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
| summarize {avg_ms = avg(duration) / 1ms, p95_ms = percentile(duration, 95) /
1ms, query_count = count()}, by:{db.system, db.namespace}
| sort p95_ms desc
```

5. Information Density by Tier

Information density describes how much data a viewer must process. Match density to the audience's time and technical depth.

Aspect	Executive	Operations	Engineering
Tiles per dashboard	4-6	8-12	10-15
Time range	7d-30d trends	1h-6h real-time	15m-1h focused
Chart complexity	Single values, simple lines	Mixed: lines, bars, tables	Tables, detailed charts
Filters/variables	None or minimal	Environment, service	Service, namespace, endpoint
Text tiles	Section headers, links to ops	Alert context, runbooks	Technical notes, query docs
Viewing time	10-30 seconds	1-5 minutes	5-30 minutes

> **Tip:** If an executive dashboard takes more than 30 seconds to understand, it has too much detail. Push that detail down to the operations tier.

6. Refresh Cadence Recommendations

Auto-refresh should match how the dashboard is consumed.

Dashboard Type	Refresh Interval	Rationale
Executive (wall screen)	10-15 min	Data changes slowly; reduce load
Executive (weekly review)	Manual	Viewed periodically, not monitored
Operations (NOC wall)	1-2 min	Real-time situational awareness

Dashboard Type	Refresh Interval	Rationale
Operations (on-call)	2-5 min	Balance freshness with query cost
Engineering (investigation)	Manual	Engineer triggers when ready
Engineering (release check)	Manual, after each deployment	Dashboards refresh on a timer or by hand — nothing triggers a refresh from a deployment

> **Important:** Aggressive refresh intervals (< 1 min) on complex queries can cause unnecessary load. Start at 5 minutes and decrease only if real-time awareness is critical.

7. Summary and Next Steps

In this notebook you learned:

- The three-tier dashboard hierarchy: Executive, Operations, Engineering
- Appropriate metrics, tile types, and complexity for each tier
- How to manage information density based on audience
- Refresh cadence recommendations that balance freshness with performance
- Sample DQL queries for each tier level

Next: In **DASH-03: Executive Dashboards**, we deep-dive into building polished executive-level dashboards with KPI selection, availability calculations, and MTTR tracking.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.